

CO5 AT1

1. **Two large Pandas DataFrames containing customer transaction and demographic data must be combined. However, constraints include: (i) no duplication of customer IDs, (ii) consistent indexing after merging, and (iii) preservation of all valid records. Describe how you will perform the combining operation, validate constraints using filtering and grouping, and ensure correctness through sorting and indexing techniques. Include steps to debug mismatched or missing entries.**

Combining Customer Transaction and Demographic Data

Introduction

In real-world applications, customer information is often stored in different DataFrames. For example, transaction data may contain customer purchases, while demographic data contains age, gender, location, etc.

Pandas provides operations such as `merge()`, `concat()`, filtering, grouping, sorting, and indexing to combine and validate these datasets.

Step 1: Create/Load the DataFrames

```
import pandas as pd
```

```
transactions = pd.read_csv("transactions.csv")
```

```
demographics = pd.read_csv("demographics.csv")
```

```
print(transactions.head())
```

```
print(demographics.head())
```

Example:

Transactions DataFrame

Customer_ID	Transaction_ID	Amount
C101	T001	500
C102	T002	750
C103	T003	300

Demographics DataFrame

Customer_ID	Age	City
C101	25	Chennai
C102	30	Hyderabad
C103	28	Bangalore

Step 2: Check Duplicate Customer IDs

```
print(transactions["Customer_ID"].duplicated().sum())  
print(demographics["Customer_ID"].duplicated().sum())
```

To identify duplicate records:

```
duplicates = demographics[
```

```
demographics["Customer_ID"].duplicated(keep=False)  
]
```

```
print(duplicates)
```

If demographic data should contain only one record per customer, duplicates can be removed:

```
demographics = demographics.drop_duplicates(  
    subset="Customer_ID",  
    keep="first"  
)
```

Step 3: Combine the DataFrames

A merge based on Customer_ID can be performed using:

```
combined = pd.merge(  
    transactions,  
    demographics,  
    on="Customer_ID",  
    how="outer"  
)
```

The outer merge is useful because it **preserves all valid records** from both DataFrames.

Other merge types include:

- inner – keeps only matching customers.
- left – keeps all transaction records.
- right – keeps all demographic records.
- outer – keeps records from both DataFrames.

Step 4: Check Missing Entries

```
print(combined.isnull().sum())
```

Find customers with missing demographic information:

```
missing_demo = combined[  
    combined["Age"].isnull() |  
    combined["City"].isnull()  
]
```

```
print(missing_demo)
```

Missing values can be handled using:

```
combined["Age"] = combined["Age"].fillna(  
    combined["Age"].median()  
)
```

```
combined["City"] =  
combined["City"].fillna("Unknown")
```

Step 5: Validate Using Filtering and Grouping

Check whether each customer has the expected number of records:

```
customer_count = combined.groupby(  
    "Customer_ID"  
)  
.size()
```

```
print(customer_count)
```

Check customers with more than one demographic record:

```
check = demographics.groupby(  
    "Customer_ID"  
)  
.size()
```

```
print(check[check > 1])
```

Filtering can also be used:

```
high_value = combined[  
    combined["Amount"] > 500  
]
```

```
print(high_value)
```

Step 6: Sort and Reset the Index

Sort the combined data:

```
combined = combined.sort_values(  
    by="Customer_ID"  
)
```

Reset the index:

```
combined = combined.reset_index(drop=True)
```

Check the final index:

```
print(combined.index)
```

This ensures a **consistent sequential index** after merging and sorting.

Step 7: Debugging Mismatched Customer IDs

Customer IDs may have different formats, such as C101 and c101, or unwanted spaces.

```
transactions["Customer_ID"] = (  
    transactions["Customer_ID"]  
    .astype(str)  
    .str.strip()  
    .str.upper()  
)
```

```
demographics["Customer_ID"] = (  
    demographics["Customer_ID"]  
    .astype(str)  
    .str.strip()  
    .str.upper()  
)
```

Find IDs present only in transactions:

```
only_transactions = set(  
    transactions["Customer_ID"]  
) - set(  
    demographics["Customer_ID"]  
)
```

```
print(only_transactions)
```

Find IDs present only in demographics:

```
only_demographics = set(  
    demographics["Customer_ID"]  
) - set(  
    transactions["Customer_ID"]  
)
```

```
print(only_demographics)
```

Result

The two DataFrames are successfully combined using `merge()`. Duplicate IDs, missing values, mismatched IDs, sorting, and indexing are checked to ensure that all valid customer records are preserved.

2.Design a complete data analysis pipeline that reads multiple data files, processes them using Pandas Data Frames, and generates visualizations. Constraints include: (i) efficient file I/O operations, (ii) correct grouping and sorting of data, (iii) filtering based on user-defined conditions, and (iv) meaningful plotting outputs. Explain how each stage (loading, processing, aggregation, and plotting) will be implemented and optimized, along with debugging steps for handling corrupted files or inconsistent data formats.

Complete Pandas Data Analysis Pipeline

Introduction

A data analysis pipeline is a sequence of steps used to load, clean, process, analyze, and visualize data. Pandas DataFrames provide an efficient way to perform these operations on multiple files.

Pipeline

Multiple Files → Loading → Cleaning → Filtering → Grouping → Sorting → Aggregation → Visualization → Results

Step 1: Import Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import glob
import os
```

Step 2: Read Multiple Files

Suppose several CSV files are stored in a folder.

```
files = glob.glob("data/*.csv")
```

```
dataframes = []
```

```
for file in files:
```

```
    df = pd.read_csv(file)
    dataframes.append(df)
```

```
data = pd.concat(
    dataframes,
    ignore_index=True
)
```

```
print(data.head())
```

Using `ignore_index=True` provides a consistent index after combining files.

Step 3: Check the Data

```
print(data.shape)
```

```
print(data.info())
print(data.head())
Check missing values:
print(data.isnull().sum())
Check duplicate records:
print(data.duplicated().sum())
Remove duplicates:
data = data.drop_duplicates()
```

Step 4: Handle Inconsistent Data

Convert columns to the correct data type:

```
data["Amount"] = pd.to_numeric(
    data["Amount"],
    errors="coerce"
)
```

Convert dates:

```
data["Date"] = pd.to_datetime(
    data["Date"],
    errors="coerce"
)
```

Remove invalid records:

```
data = data.dropna(
    subset=["Amount", "Date"]
)
```

Step 5: Apply User-Defined Filtering

For example, find transactions above ₹500:

```
filtered_data = data[
    data["Amount"] > 500
]
```



```
print(filtered_data)
```

Multiple conditions can be applied:

```
filtered_data = data[  
    (data["Amount"] > 500) &  
    (data["City"] == "Chennai")  
]
```

This allows the user to analyze only the required records.

Step 6: Group the Data

Calculate total sales for each city:

```
city_sales = data.groupby(  
    "City"  
)["Amount"].sum()
```

```
print(city_sales)
```

Calculate average transaction amount:

```
average_sales = data.groupby(  
    "City"  
)["Amount"].mean()
```

```
print(average_sales)
```

Step 7: Sort the Results

Sort cities according to total sales:

```
city_sales = city_sales.sort_values(  
    ascending=False  
)
```

```
print(city_sales)
```

For a DataFrame:

```
result = data.sort_values(  
    ascending=False  
)
```

```
by="Amount",  
ascending=False  
)
```

Step 8: Generate Visualizations

Bar Chart

```
city_sales.plot(  
    kind="bar",  
    title="Total Sales by City"  
)
```

```
plt.xlabel("City")  
plt.ylabel("Total Sales")  
plt.show()
```

Line Plot

```
daily_sales = data.groupby(  
    "Date"  
)["Amount"].sum()
```

```
daily_sales.plot(  
    kind="line",  
    title="Daily Sales"  
)
```

```
plt.xlabel("Date")  
plt.ylabel("Sales")  
plt.show()
```

Histogram

```
data["Amount"].plot(  
    kind="hist",  
    bins=10,
```

```
    title="Transaction Amount Distribution"  
)
```

```
plt.xlabel("Amount")  
plt.show()
```

Scatter Plot

```
plt.scatter(  
    data["Age"],  
    data["Amount"]  
)
```

```
plt.xlabel("Age")  
plt.ylabel("Transaction Amount")  
plt.title("Age vs Transaction Amount")  
plt.show()
```

Step 9: Debugging Corrupted or Inconsistent Files

Problem 1: File cannot be read

try:

```
    df = pd.read_csv(file)  
except Exception as e:  
    print("Error reading:", file)  
    print(e)
```

Problem 2: Different column names

```
df.columns = (  
    df.columns  
    .str.strip()  
    .str.lower()  
)
```

Problem 3: Different data types

```
df["Amount"] = pd.to_numeric(  

```

```
df["Amount"],  
errors="coerce"  
)
```

Problem 4: Missing values

```
print(df.isnull().sum())
```

Problem 5: Duplicate records

```
print(df.duplicated().sum())
```

Optimization Techniques

1. **Read only required columns** to reduce memory usage.

```
df = pd.read_csv(  
    "sales.csv",  
    usecols=["Customer_ID", "Amount", "City"]  
)
```

2. **Use appropriate data types** for better memory efficiency.
3. **Remove unnecessary duplicate and invalid records** early.
4. **Use vectorized Pandas operations** instead of unnecessary loops.
5. **Group and aggregate only required columns.**
6. **Use ignore_index=True** when concatenating multiple files.

Final Conclusion

Pandas provides an efficient framework for combining, cleaning, filtering, grouping, sorting, and analyzing large datasets. The complete pipeline ensures that data from multiple files is processed correctly, while validation and

debugging techniques help identify missing, duplicate, corrupted, or inconsistent records. Finally, meaningful visualizations such as bar charts, line plots, histograms, and scatter plots make the analyzed results easier to understand.